TD 2 – – Fiche

Guillaume THÉMÈZE – Ingénieur
Enseignant contact@themeze.fr

Plan de la séance

Durée : 🕒=15 min	—	🕒🕒=30 min	—	🕒🕒🕒=45 min	—	🕒🕒🕒🕒=60 min	—	🕒🕒🕒🕒🕒=75 min
Difficulté : ⋆=très facile	—	⋆⋆=facile	—	⋆⋆⋆=moyen	—	⋆⋆⋆⋆=difficile	→	⋆⋆⋆⋆⋆=très difficile
Priorité : ⚐=1	—	⚐⚐=2	—	⚐⚐⚐=3	—	⚐⚐⚐⚐=4	→	⚐⚐⚐⚐⚐=5
Statut : ○ facultatif	—		—		—		—	⊙ essentiel

no	**Exercice**	**Type**	**Diff.**	**Priorité**	**Durée**	**Statut**
1	Listes : génération d'un vecteur temps régulier	Application directe	⋆	⚐	🕒	⊙ **essentiel**
2	Fonction `get_list` et mesure du temps d'exécution	Application directe	⋆	⚐	🕒	⊙ **essentiel**
3	Génération d'un vecteur avec Numpy : vectorisation	Application directe	⋆	⚐	🕒	⊙ **essentiel**
4	Génération de signaux et affichage avec matplotlib	Application longue, cœur du bloc	⋆⋆⋆	⚐	🕒🕒🕒	⊙ **essentiel**
5	Fonction porte : indexation booléenne Numpy	Application directe	⋆⋆	⚐	🕒	⊙ **essentiel**
6	Matrices Numpy : indexation, axes et pièges de dtype	Application longue	⋆⋆⋆	⚐	🕒🕒🕒	⊙ **essentiel**
7	Équation du second degré : réels puis complexes *(facultatif)*	Exercice facultatif, fonctions, conditions et complexes	⋆⋆	⚐	🕒🕒	○ facultatif

Durée totale estimée : 160 min (dont 135 min pour les exercices essentiels ; 1 exercice facultatif).

➡ • 10 min

Exercice 1. Listes : génération d'un vecteur temps régulier

★☆☆☆☆

Application directe

◁ Plan de la séance

1. Générer une liste `L_temps` de 101 points régulièrement répartis entre 0 et 1 s.
2. Afficher la taille de cette liste.
3. Afficher le dernier élément de la liste.
4. Quelle est la période d'échantillonnage ?

Compréhensions de liste
Prérequis

À ce stade de la progression, Numpy n'a pas encore été introduit (il arrive à l'exercice 3) : l'attendu est donc une liste Python « pure », construite par exemple avec une compréhension de liste `[expr for i in range(n)]`, pas encore un tableau Numpy.

Compter intervalles, pas des points

Retenir que N points régulièrement répartis sur un intervalle définissent $N-1$ intervalles : le pas vaut donc $(\text{stop} - \text{start})/(N-1)$, pas $/N$.

Ancrer un piège récurrent du bloc

pédagogiques

Le décompte des intervalles (N points $\to$ $N-1$ intervalles) revient à chaque exercice de génération de vecteur du bloc (`td2_ex2`, `td2_ex3`...) : bien l'ancrer ici, dès le premier exercice.

Solution commentée

Point de cours

```
nombre_de_points = 101
pas = 1 / (nombre_de_points - 1)   # 1/100 = 0.01, PAS 1/101

L_temps = [i * pas for i in range(nombre_de_points)]

print("Taille de L_temps :", len(L_temps))
print("Dernier element de L_temps :", L_temps[-1])

Te = L_temps[1] - L_temps[0]
print(f"Periode d'echantillonnage Te = {Te} s")
```

Résultats : `len(L_temps) = 101` ; `L_temps[-1] = 1.0` ; `Te = 0.01 s`.

Précision flottante

Point de cours

Avec des flottants, `L_temps[k+1] - L_temps[k]` peut varier de quelques 10^{-17} d'un couple de points à l'autre à cause des arrondis binaires : c'est normal et sans conséquence pratique — bonne occasion d'introduire la précision `float64` si des étudiants s'en étonnent.

1. Générer `L_temps`, puis afficher sa taille et son dernier élément.

Combien d'intervalles y a-t-il entre 101 points ?

Construction et vérification

Correction

`len(L_temps)` = 101 ; `L_temps[-1]` = 1.0.

Solution détaillée

Avec `pas = 1/100`, la compréhension de liste `[i * pas for i in range(101)]` produit bien 101 valeurs allant de 0 à $100 \times 0{,}01 = 1{,}0$ inclus.

2. Quelle est la période d'échantillonnage T_e de cette liste ?

T_e est l'écart entre deux points consécutifs.

Calcul de Te

Correction

Te = 0.01 s.

Solution détaillée

$T_e = L\_temps[1] - L\_temps[0] = 0{,}01$ s. On peut aussi la lire directement dans le code (variable pas), mais la retrouver à partir de la liste elle-même est une bonne pratique si l'on ne connaît pas la formule de génération (utile par exemple si la liste provient d'un fichier de données).

3. Si l'on voulait un pas de 5 ms, combien de points faudrait-il pour couvrir 1 seconde ?

1 seconde / 5 ms donne le nombre d'intervalles, pas directement le nombre de points.

Correction

201 points (200 intervalles de 5 ms).

Calcul

Un pas de 5 ms sur 1 s correspond à $1/0{,}005 = 200$ intervalles, donc $200 + 1 = 201$ points.

`pas = 1/101` au lieu de `1/100` fait que le dernier point vaut environ 0,99 au lieu de 1,0 : la liste ne couvre pas tout l'intervalle demandé. Faire compter les intervalles sur un petit exemple (3 points sur [0,1] → 2 intervalles de 0,5) avant de généraliser à 101 points.

`range(0, 1, pas)` lève une `TypeError` : `range` n'accepte que des entiers, ce n'est pas un outil de génération de valeurs réelles.

Le seul vrai piège est le décompte des intervalles ; ne pas s'y attarder au-delà du temps nécessaire pour le faire bien comprendre à toute la salle.

Bilan

Une règle à retenir pour tout le bloc : générer N points régulièrement répartis entre `start` et `stop` nécessite un pas de $(\text{stop} - \text{start})/(N-1)$.

Exercice 2. Fonction `get_list` et mesure du temps d'exécution

• 15 min

Application directe

◃ Plan de la séance

1. Créer une fonction `get_list` qui génère une liste de N nombres réels régulièrement répartis entre une valeur `start` et une valeur `stop`. N, `start` et `stop` seront des paramètres de cette fonction. `start` aura une valeur par défaut de 0.
2. Mesurer le temps d'exécution de la création d'une liste `L_test` de 10 000 points répartis entre 0 et 1.

En Python, `def f(a, b, c=0):` définit un paramètre `c` optionnel : si l'appelant ne le fournit pas, la valeur 0 est utilisée.

Respecter l'ordre de paramètres demandé par l'énoncé (`N, stop, start=0`) plutôt que de le deviner par analogie avec `np.linspace(start, stop, num)`, qui utilise un ordre différent.

Utiliser `time.perf_counter()` en encadrant strictement la portion de code à mesurer, sans y inclure l'affichage ni les imports.

Ne pas s'attarder sur la valeur numérique du chronométrage : ce qui compte est la méthode de mesure, qui sera immédiatement réutilisée à l'identique dans l'exercice suivant (`td2_ex3`) pour la comparaison liste Python / vecteur Numpy.

Solution commentée
Point de cours

```
import time

def get_list(N: int, stop: float, start: float = 0) -> list[float]:
    """Genere une liste de N reels regulierement repartis
    entre start et stop (bornes incluses)."""
    # N points -> N - 1 intervalles (meme piege qu'a l'exercice 1).
    step = (stop - start) / (N - 1)
    return [start + i * step for i in range(N)]


# time.perf_counter() : plus precis que time.time(), et insensible
# aux ajustements de l'horloge systeme. On n'encadre QUE l'appel
# de la fonction, ni l'affichage ni l'import.
debut = time.perf_counter()
L_test = get_list(10_000, 1)
fin = time.perf_counter()

duree_liste_python = fin - debut
print(f"Temps d'execution (liste Python, 10000 points) : "
      f"{duree_liste_python:.6f} s")
```

Ordre de grandeur observé (dépend fortement de la machine) : environ $0{,}0005$ à $0{,}001$ s pour 10 000 points.

1. Écrire `get_list(N, stop, start=0)` en respectant l'ordre de paramètres imposé par l'énoncé.

oup de pou

Le pas se calcule comme à l'exercice précédent : N points, $N-1$ intervalles.

Correction

step = (stop - start) / (N - 1) ; return [start + i*step for i in range(N)]

Implémentation
Solution détaillée

```
def get_list(N, stop, start=0):
    step = (stop - start) / (N - 1)
    return [start + i * step for i in range(N)]
```

Test rapide : `get_list(5, 1)` avec start par défaut (0), et `get_list(5, 10, 5)` entre 5 et 10.

2. Mesurer le temps d'exécution de la génération de 10 000 points entre 0 et 1 avec cette fonction.

oup de pou

Encadrez uniquement l'appel de `get_list`, pas l'`import time` ni le `print` du résultat.

Correction

Environ 0.0005 à 0.001 s pour 10 000 points (ordre de grandeur, dépend de la machine).

Mesure
Solution détaillée

```
debut = time.perf_counter()
L_test = get_list(10_000, 1)
fin = time.perf_counter()
duree_liste_python = fin - debut
```

La valeur absolue mesurée varie beaucoup d'une machine à l'autre : ce qui compte est l'ordre de grandeur, pour la comparaison avec la version Numpy de l'exercice suivant.

3. Pourquoi utiliser `time.perf_counter()` plutôt que `time.time()` ?

Coup de pouce

L'un des deux peut être affecté par une resynchronisation de l'horloge système (ex. NTP) pendant la mesure.

Correction

`perf_counter()` est plus précis et insensible aux ajustements de l'horloge système.

Réponse

Solution détaillée

`time.time()` renvoie l'heure « murale » du système, qui peut être modifiée pendant l'exécution (synchronisation réseau, changement de fuseau…), ce qui fausserait une mesure de durée. `time.perf_counter()` utilise une horloge monotone dédiée aux mesures de performance, avec une meilleure résolution : c'est le choix recommandé pour chronométrer un morceau de code.

Inclure l'affichage (`print`) ou l'import dans le chronométrage rend la mesure incomparable avec celle de l'exercice suivant : n'encadrer de `perf_counter()` que la ligne d'appel de la fonction elle-même.

Même remède qu'à l'exercice précédent : dessiner un petit exemple pour fixer la formule du pas.

Bilan

`time.perf_counter()` est le bon outil pour mesurer un temps d'exécution : plus précis que `time.time()`, non affecté par un changement d'horloge système.

➔ • 15 min

Exercice 3. Génération d'un vecteur avec Numpy : vectorisation

★☆☆☆☆

Application directe

◁ Plan de la séance

1. Importer la bibliothèque Numpy.
2. Générer un vecteur `V_test` de 10 000 nombres réels répartis entre 0 et 1 avec Numpy.
3. Afficher la taille de ce vecteur.
4. Afficher le dernier élément de ce vecteur.
5. Mesurer le temps d'exécution de cette génération.
6. Comparer à la mesure de l'exercice précédent. Que concluez-vous ?

Prérequis

`np.linspace(start, stop, num)` génère `num` valeurs régulièrement réparties entre `start` et `stop` (bornes incluses) : c'est l'équivalent Numpy, vectorisé, de la fonction `get_list` écrite à la main à l'exercice précédent.

Formuler explicitement la raison de l'écart de performance : vectorisation, exécution en code C compilé, absence de boucle Python interprétée élément par élément.

Préférer `.size` (nombre total d'éléments) à `len()` sur un tableau Numpy : `.size` reste correct en 2D et plus, contrairement à `len()` qui ne renverrait que la taille de la première dimension.

Résultat clé du bloc

Ce constat (Numpy vectorisé bat une boucle Python) sera réutilisé et amplifié en séance 3 (comparaison boucle / meshgrid) : insister sur le principe général plutôt que sur la valeur numérique précise du facteur d'accélération, qui varie beaucoup d'une machine à l'autre.

Guillaume THÉMÈZE <contact@themeze.fr>

Solution commentée
Point de cours

```
import time
import numpy as np   # alias quasi universel

V_test = np.linspace(0, 1, 10_000)

print("Taille de V_test (.size) :", V_test.size)
print("Forme de V_test (.shape) :", V_test.shape)
print("Dernier element de V_test :", V_test[-1])

debut = time.perf_counter()
V_test_mesure = np.linspace(0, 1, 10_000)
fin = time.perf_counter()
duree_numpy = fin - debut
print(f"Temps d'execution (Numpy, 10000 points) : {duree_numpy:.6f} s")
```

Comparaison et conclusion
Point de cours

```
print(f"Rappel - temps liste Python (boucle)   : {duree_liste_python:.6f} s")
print(f"Temps vecteur Numpy (vectorise)         : {duree_numpy:.6f} s")
print(f"Facteur d'acceleration approximatif     : "
      f"x{duree_liste_python / duree_numpy:.0f}")
```

Mesure typique observée (une exécution) : liste Python $\approx 0{,}00067$ s, Numpy $\approx 0{,}00007$ s, soit un facteur $\approx \times 10$ (peut varier de $\times 5$ à $\times 100$ selon la machine). Conclusion attendue : Numpy est plus rapide car vectorisé (code C, pas de boucle Python explicite, pas d'overhead d'interpréteur à chaque élément).

1. Générer `V_test = np.linspace(0, 1, 10_000)`. Afficher sa taille (`.size`), sa forme (`.shape`) et son dernier élément.

oup de pou

`.size` donne le nombre total d'éléments ; `.shape` donne la forme du tableau sous forme de tuple.

Correction

`V_test.size` = 10000 ; `V_test.shape` = (10000,) ; `V_test[-1]` = 1.0.

Résultat
Solution détaillée

`np.linspace(0, 1, 10_000)` génère 10 000 valeurs entre 0 et 1 (bornes incluses), donc `.size` vaut $10\,000$, `.shape` vaut $(10\,000,)$ et le dernier élément vaut exactement $1{,}0$.

2. Mesurer le temps d'exécution de cette génération et le comparer à celui de la fonction `get_list` de l'exercice précédent. Conclure.

Coup de pouce

Recalculez si besoin `duree_liste_python` dans le même script, pour comparer sur la même exécution.

Correction

Numpy est nettement plus rapide (souvent x10 à x100) : vectorisation, code C compilé.

Comparaison
Solution détaillée

Sur une exécution typique : liste Python $\approx 0{,}00067$ s, vecteur Numpy $\approx 0{,}00007$ s, soit un facteur d'environ $\times 10$ (le facteur exact varie fortement d'une machine à l'autre, parfois de $\times 5$ à $\times 100$). La cause : une boucle Python réévalue chaque itération une par une (overhead de l'interpréteur CPython à chaque tour), alors que `np.linspace` est implémenté en C et exécute le calcul de façon vectorisée, en une seule opération sur tout le tableau.

3. Le facteur d'accélération serait-il le même avec $N = 10$ ou $N = 1\,000\,000$?

oup de pou

Un tableau Numpy a un coût fixe de création, indépendant de N.

Correction

Non : pour un petit N le coût fixe de création domine, Numpy peut même être plus lent.

Discussion
Solution détaillée

Pour un très petit N (quelques dizaines de points), le coût fixe de création de l'objet `ndarray` peut dominer le calcul lui-même : Numpy peut alors se révéler plus lent qu'une simple liste Python. À l'inverse, pour un très grand N (par ex. $1\,000\,000$), le coût par élément de la boucle Python domine largement le coût fixe de Numpy, et le facteur d'accélération devient encore plus favorable à Numpy qu'avec $N = 10\,000$.

Conclure sans expliquer

Se contenter de « Numpy est plus rapide » sans expliquer pourquoi : faire formuler explicitement vectorisation, code compilé C, absence de boucle interprétée.

Comparaison biaisée

Vérifier que les deux mesures (liste et Numpy) portent strictement sur la même tâche : même N, même intervalle, aucune opération annexe incluse dans le chronométrage.

Généraliser trop vite
pédagogiques (écueils

Pour de très petits N (quelques dizaines de points), Numpy peut être plus lent que la liste Python à cause du coût fixe de création de l'objet `ndarray` : ne pas généraliser « Numpy toujours plus rapide » sans préciser « à partir d'une certaine taille ».

Vectorisation Numpy
Bilan

Préférer les opérations Numpy vectorisées aux boucles Python explicites dès que l'on manipule de grands volumes de données numériques : le gain de performance devient net à partir de quelques milliers d'éléments.

➔ • 45 min

Exercice 4. Génération de signaux et affichage avec matplotlib

★★★☆☆

Application longue, cœur du bloc

◁ Plan de la séance

1. Créer un vecteur temps de 101 points régulièrement répartis entre 0 et 1 s. Quelle est la période d'échantillonnage ?
2. Importer `matplotlib.pyplot`.
3. Tracer une sinusoïde de période 50 ms en rouge, avec titre, axes et légende.
4. Que pensez-vous du résultat ? Améliorer le résultat.
5. Tracer sur le même graphique une sinusoïde de période 20 ms en bleu.
6. Faire un zoom pour n'afficher que quelques périodes des deux sinusoïdes.
7. Faire une fonction générant un signal sinusoïdal sur 10 périodes avec une fréquence d'échantillonnage 6 fois supérieure à la fréquence du signal. Arguments : période et amplitude.
8. Tester cette fonction avec une période de 30 ms et afficher le signal.
9. Compléter la fonction pour qu'elle prenne un vecteur temps en argument supplémentaire (défaut : liste vide). Si vide, même comportement qu'avant ; sinon, générer le signal sur ce vecteur. Message si la fréquence d'échantillonnage n'est pas correcte.
10. Tester cette nouvelle fonctionnalité.

Critère de Nyquist vs confort visuel
Prérequis

Le critère de Nyquist ($f_e > 2f$) est un minimum théorique de *reconstruction* du signal. Il est distinct du critère de *confort visuel* d'un tracé matplotlib, qui nécessite bien plus de points par période (typiquement 10 à 100) pour ne pas donner un rendu anguleux — les deux notions sont vues successivement dans cet exercice (questions 4 et 9).

Ajouter systématiquement titre, labels d'axes, légende et grille à un tracé matplotlib ; utiliser `plt.xlim` pour zoomer sans recalculer les données.

Ajouter un paramètre par défaut à une fonction déjà écrite, et gérer deux comportements distincts selon que cet argument est fourni ou non.

Les questions 7 à 10 (fonction avec argument optionnel et message conditionnel) préparent directement le travail sur les fonctions de la séance 3 (FFT, meshgrid). Prévoir de circuler activement pendant les questions 3-4, le point le plus visuel et le plus fréquemment source de blocage.

Questions 1 à 6 : tracé et zoom (extrait commenté)

Point de cours

```
import numpy as np
import matplotlib.pyplot as plt

time_vector = np.linspace(0, 1, 101)
Te = time_vector[1] - time_vector[0]     # 0.01 s

periode_1 = 0.050            # 50 ms
frequence_1 = 1 / periode_1  # 20 Hz
signal_1 = np.sin(2 * np.pi * frequence_1 * time_vector)

plt.figure()
plt.plot(time_vector, signal_1, color="red",
         label=f"Periode = {periode_1*1000:.0f} ms")
plt.xlabel("Temps (s)"); plt.ylabel("Amplitude")
plt.title("Sinusoide de periode 50 ms")
plt.legend(); plt.grid()
plt.show()

# Q4 : avec Te=10 ms pour une periode de 50 ms, seulement 5 points
# par periode -> rendu anguleux. On regenere un vecteur temps plus
# fin, SANS changer la duree totale du signal.
time_vector_fin = np.linspace(0, 1, 2001)   # Te ~= 0.5 ms
signal_1_fin = np.sin(2 * np.pi * frequence_1 * time_vector_fin)

# Q5-6 : deuxieme sinusoide + zoom avec xlim (ne recalcule rien,
# restreint seulement la fenetre d'affichage).
periode_2 = 0.020
frequence_2 = 1 / periode_2
signal_2_fin = np.sin(2 * np.pi * frequence_2 * time_vector_fin)

plt.figure()
plt.plot(time_vector_fin, signal_1_fin, color="red", label="50 ms")
plt.plot(time_vector_fin, signal_2_fin, color="blue", label="20 ms")
plt.xlim(0, 0.1)   # zoom : 2 periodes de la rouge, 5 de la bleue
plt.legend(); plt.grid(); plt.show()
```

Questions 7-8 : fonction `generate_sine_10_periods`

Point de cours

```
def generate_sine_10_periods(period: float, amplitude: float):
    """Genere un signal sinusoidal sur 10 periodes, echantillonne
    a une frequence 6 fois superieure a la frequence du signal."""
    frequency = 1 / period
    sampling_frequency = 6 * frequency
    Te = 1 / sampling_frequency
    duration = 10 * period

    # np.arange(0, duration, Te) : le nombre de points decoule de
    # Te et duration (contrairement a linspace, ou l'on fixerait
    # le nombre de points a priori).
    time_vector = np.arange(0, duration, Te)
    signal = amplitude * np.sin(2 * np.pi * frequency * time_vector)
    return time_vector, signal

t_30ms, sig_30ms = generate_sine_10_periods(period=0.030, amplitude=2)
```

Avec seulement 6 points par période, le rendu est très anguleux (utiliser `marker="o"` pour bien visualiser les points réels) : c'est volontaire ($f_e = 6f$), et un bon écho à la question 4 (6 points/période respecte Nyquist mais reste visuellement grossier).

Question 9-10 : version avec vecteur temps optionnel

Point de cours

```
def generate_sine_10_periods_v2(
    period: float,
    amplitude: float,
    time_vector: np.ndarray | list = [],   # cf. piege ci-dessous
):
    frequency = 1 / period

    # len(...) == 0 fonctionne pour une liste vide ET un ndarray
    # vide, contrairement a "time_vector == []" qui leverait une
    # erreur avec un ndarray non vide (comparaison ambigue).
    if len(time_vector) == 0:
        sampling_frequency = 6 * frequency
        Te = 1 / sampling_frequency
        duration = 10 * period
        time_vector = np.arange(0, duration, Te)
    else:
        time_vector = np.asarray(time_vector)
        # Frequence d'echantillonnage deduite du vecteur FOURNI.
        Te_fourni = np.mean(np.diff(time_vector))
        sampling_frequency = 1 / Te_fourni

        # Critere de Nyquist strict : fe > 2f.
        if sampling_frequency <= 2 * frequency:
            print(
                f"ATTENTION : frequence d'echantillonnage "
                f"insuffisante (fe={sampling_frequency:.1f} Hz, "
                f"signal a {frequency:.1f} Hz)."
            )

    signal = amplitude * np.sin(2 * np.pi * frequency * time_vector)
    return time_vector, signal
```

1. Créer le vecteur temps de 101 points sur [0, 1] s et calculer sa période d'échantillonnage.

oup de pou

Même raisonnement que `td2_ex1` : N points, $N-1$ intervalles.

Correction

`time_vector = np.linspace(0,1,101)` ; Te = 0.01 s.

Calcul

Solution détaillée

`time_vector = np.linspace(0, 1, 101)` donne `Te = time_vector[1] - time_vector[0] = 0,01` s.

2. Tracer la sinusoïde de période 50 ms sur ce vecteur temps, en rouge, avec titre, axes et légende. Que pensez-vous du résultat ? Comment l'améliorer ?

oup de pou

Combien de points par période cela fait-il, avec $T_e = 10$ ms et une période de 50 ms ?

Correction

Seulement 5 points/période : rendu anguleux. Régénérer un vecteur temps plus fin, sans changer la durée.

Diagnostic et amélioration

Solution détaillée

Avec $T_e = 10$ ms pour une période de 50 ms, on n'a que $50/10 = 5$ points par période : la courbe reliée par segments de droite a un aspect anguleux, peu représentatif d'une vraie sinusoïde. Ce n'est pas un sous-échantillonnage au sens de Nyquist ($5 > 2$), mais c'est insuffisant pour un rendu visuel lisse. L'amélioration consiste à augmenter le nombre de points du vecteur temps (par ex. `np.linspace(0, 1, 2001)`), sans changer la durée totale du signal.

3. Écrire `generate_sine_10_periods(period, amplitude)`, qui génère un signal sur 10 périodes avec $f_e = 6f$. La tester avec une période de 30 ms.

oup de pou

`duration = 10 * period`; `Te = 1 / (6 * frequency)`.

Correction

np.arange(0, 10*period, 1/(6/period)) — voir code ci-dessus.

Implémentation

Solution détaillée

Voir le code commenté ci-dessus (« Questions 7-8 »). Avec `period=0.030`, `amplitude=2`, on obtient un signal de 6 points par période, sur 10 périodes, dont le rendu très anguleux illustre concrètement l'écart entre le critère de Nyquist et un affichage visuellement satisfaisant.

4. Compléter la fonction pour accepter un vecteur temps optionnel (défaut : liste vide). Si un vecteur est fourni, en déduire f_e et afficher un message si le critère de Nyquist n'est pas respecté. Tester avec un vecteur bien échantillonné, puis un vecteur trop grossier.

Coup de pouce

Utilisez `np.mean(np.diff(time_vector))` pour déduire T_e d'un vecteur temps quelconque (supposé régulier).

Correction

Voir `generate_sine_10_periods_v2` ci-dessus ; le cas « mal échantillonné » doit afficher le message d'alerte.

Validation

Solution détaillée

Avec `t_manuel_ok = np.linspace(0, 0.3, 1000)`, largement suffisant pour 30 ms, aucun message n'apparaît. Avec `t_manuel_ko = np.linspace(0, 0.3, 5)`, bien trop peu de points, le message d'avertissement doit s'afficher : c'est le test de compréhension à faire vérifier avant de considérer l'exercice terminé.

Confondre Nyquist et rendu visuel

pédagogiques (écueils

Considérer que 5 points par période (question 3-4) est un bug de sous-échantillonnage Nyquist : 5 points reste au-dessus du minimum théorique de 2 (Nyquist respecté), mais insuffisant pour un affichage lisse. Bien distinguer les deux notions.

Piège de l'argument par défaut mutable

Remarques pédagogiques (écueils classiques)

`time_vector=[]` est un piège Python classique : une valeur par défaut *mutable* (liste, dict...) est créée une seule fois à la définition de la fonction et partagée entre tous les appels qui ne fournissent pas cet argument. Ici, sans conséquence car la fonction réaffecte `time_vector` sans le modifier en place — bon prétexte pour montrer le piège avec un contre-exemple :

```
def ajoute(valeur, ma_liste=[]):
    ma_liste.append(valeur)
    return ma_liste
print(ajoute(1))   # [1]
print(ajoute(2))   # [1, 2]  <- surprise !
```

Fonctionne pour une liste vide, mais lève une `ValueError` (« ambiguous truth value ») si un tableau Numpy non vide est passé. Utiliser `len(time_vector) == 0`.

Réutiliser `fe = 6 frequency` codée en dur au lieu de recalculer f_e à partir du vecteur temps réellement fourni par l'utilisateur : le message d'alerte de la question 9 n'a de sens que si f_e est bien recalculée.

Deux critères, deux échelles

Bilan

Nyquist ($f_e > 2f$) garantit qu'un signal *peut* être reconstruit ; un rendu visuel lisse en demande bien davantage. Bien distinguer les deux avant de conclure qu'un tracé « cassé » est forcément un bug de sous-échantillonnage au sens strict.

➔ • 15 min

Exercice 5. Fonction porte : indexation booléenne Numpy

★★☆☆☆

Application directe

◁ Plan de la séance

1. Créer une fonction `porte` qui renvoie un vecteur de booléens à partir d'un vecteur `v_in` : vrai si `v_in` est compris entre `min` et `max`, faux sinon. `v_in`, `min` et `max` sont des paramètres.
2. Tester cette fonction.

Les opérateurs Numpy bit à bit

Prérequis

Les opérateurs Python `and`/`or`/`not` n'opèrent que sur un seul booléen. Sur un tableau Numpy de plusieurs éléments, il faut utiliser les opérateurs bit à bit `&` (et), `|` (ou) et `~` (non), qui s'appliquent élément par élément.

Construire un masque booléen combinant deux comparaisons avec `&`, en parenthésant chaque comparaison individuellement.

Prévenir un piège Numpy classique

Exercice court mais dense en pièges (`and`/`&`, priorité des opérateurs) qui reviendront dans tout usage ultérieur de Numpy : bien insister ici plutôt que de laisser les étudiants buter dessus plus tard sans en comprendre l'origine.

Solution commentée

Point de cours

```
import numpy as np

def porte(v_in: np.ndarray, v_min: float, v_max: float) -> np.ndarray:
    """Renvoie un vecteur de booleens : True la ou
    v_min <= v_in <= v_max, False ailleurs."""
    # (v_in >= v_min) cree un tableau de booleens, teste ELEMENT
    # PAR ELEMENT sur tout le vecteur, sans boucle explicite.
    # L'operateur logique ET s'ecrit '&' (pas 'and').
    return (v_in >= v_min) & (v_in <= v_max)



v_in = np.linspace(-5, 5, 11)
resultat = porte(v_in, -2, 3)

print("v_in     :", v_in)
print("porte    :", resultat)
print("Elements retenus :", v_in[resultat])   # indexation booleenne
```

Sur `v_in = np.linspace(-5, 5, 11)`, `porte(v_in, -2, 3)` renvoie `True` pour les valeurs de -2 à 3 incluses.

1. Écrire `porte(v_in, v_min, v_max)` et la tester sur `v_in = np.linspace(-5, 5, 11)` avec `v_min=-2`, `v_max=3`.

Coup de pouce

Combinez deux comparaisons Numpy avec l'opérateur bit à bit adapté, en parenthésant chaque comparaison.

Correction

```
return (v_in >= v_min) & (v_in <= v_max)
```

Implémentation et test

Solution détaillée

`porte(v_in, -2, 3)` renvoie `True` pour les éléments de -2 à 3 inclus, `False` ailleurs. On peut récupérer directement les valeurs concernées avec `v_in[porte(v_in, -2, 3)]` (indexation booléenne).

2. Que se passe-t-il si l'on utilise `and` au lieu de `&` ? Pourquoi cette erreur n'apparaît-elle pas avec un seul flottant ?

Coup de pouce

`and` exige de convertir chaque opérande en un seul booléen.

Correction

ValueError : ambiguïté de la valeur de vérité d'un tableau de plusieurs éléments.

Explication

Solution détaillée

`and` tente de convertir chacun de ses deux opérandes en un unique `bool` Python. Pour un seul flottant, la comparaison renvoie un unique booléen, sans ambiguïté. Pour un tableau Numpy de plusieurs éléments, la comparaison renvoie un tableau de plusieurs booléens : Python ne sait pas lequel choisir pour représenter « la » valeur de vérité de tout le tableau, d'où l'exception `ValueError`.

3. Comment écrire l'inverse de la fonction porte (vrai en dehors de l'intervalle) ?

Coup de pouce

L'opérateur `~` inverse un tableau de booléens.

Correction

`~porte(v_in, v_min, v_max)`, ou `(v_in < v_min) | (v_in > v_max)`.

Deux écritures équivalentes

Solution détaillée

On peut soit inverser le résultat de porte avec `~porte(v_in, v_min, v_max)`, soit écrire directement la condition complémentaire : `(v_in < v_min) | (v_in > v_max)`. Les deux expressions sont équivalentes par les lois de De Morgan.

Utiliser `and` au lieu de &

pédagogiques (écueils classiques)

`(v_in >= v_min) and (v_in <= v_max)` lève `ValueError: the truth value of an array with more than one element is ambiguous` : Python ne sait pas convertir un tableau de plusieurs booléens en un seul booléen avec `and`/`or`. Cette erreur n'apparaît pas avec un seul flottant, seulement avec un tableau de plusieurs éléments — bonne occasion d'expliquer la différence entre opérateurs logiques Python et opérateurs Numpy bit à bit.

Oubli des parenthèses

pédagogiques (écueils

`&` a une priorité plus élevée que `>=` en Python : sans parenthèses autour de chaque comparaison (`v_in >= v_min & v_in <= v_max`), le résultat est faux ou provoque une erreur. Toujours parenthéser chaque comparaison individuellement avant de les combiner avec `&` ou `|`.

and/or ne fonctionnent pas sur des tableaux

Bilan

Sur un tableau Numpy de plusieurs éléments, utiliser `&`/`|` (jamais `and`/`or`), avec des parenthèses explicites autour de chaque comparaison individuelle.

Exercice 6. Matrices Numpy : indexation, axes et pièges de dtype

➜ • 35 min ★★★☆☆

Application longue

◁ Plan de la séance

Soit la matrice $A = \begin{pmatrix} 4 & 6 & -2 & 3 \\ 2 & -1 & 0 & 1 \\ -7 & 0 & 1 & 12 \end{pmatrix}$.

1. Créer `A` avec `np.array` et l'afficher.
2. Afficher le type de `A`, le type d'un élément, la taille de `A`.
3. Afficher la seconde colonne puis la troisième ligne.
4. Afficher la moyenne de tous les éléments, par colonne, par ligne.
5. Affichage formaté : « la moyenne de tous les éléments vaut ... ».
6. Modifier `A` : 2 premières lignes ×2, puis dernière colonne /3. Quel est le type des éléments de `A` alors ?
7. Créer `B` (3×4, que des 2), remplacer sa première colonne par 0.
8. Que représente `A*B` ? Comment faire un produit matriciel entre `A` et `B` ?
9. Créer `CC` : éléments de `A*B` supérieurs à 5 conservés, sinon 0.

Indexation, type, dtype, axis

Prérequis

`A.shape` donne (nombre de lignes, nombre de colonnes) ; `A.dtype` donne le type des nombres stockés (indépendant du type Python de l'objet `A` lui-même, qui est toujours `numpy.ndarray`). Pour `np.mean(A, axis=k)`, l'axe `axis=0` *parcourt les lignes* (résultat par colonne), `axis=1` *parcourt les colonnes* (résultat par ligne).

Distinguer type de l'objet et type de données

pédagogiques (écueils)

Ne pas confondre `type(A)` (toujours `numpy.ndarray`) et `A.dtype` (le type des nombres stockés, ex. `int64` ou `float64`).

Distinguer produit élément par élément et produit matriciel

pédagogiques (écueils)

`*` est toujours terme à terme sur des tableaux Numpy ; le produit matriciel au sens de l'algèbre linéaire s'écrit avec `@` (ou `np.dot` / `np.matmul`).

Le piège de troncature entière, point le plus formateur

Objectifs pédagogiques (enseigna

Diviser une colonne d'un tableau Numpy d'**entiers** et réassigner le résultat *dans ce même tableau* tronque silencieusement le résultat, sans erreur ni avertissement. Préparer un contre-exemple où la troncature change visiblement le résultat (par ex. diviser par 5 au lieu de 3), à montrer en direct si personne dans la salle ne l'a remarqué spontanément.

Questions 1 à 5 : création, type, indexation, moyennes

Point de cours

```
import numpy as np

A = np.array([[4, 6, -2, 3], [2, -1, 0, 1], [-7, 0, 1, 12]])
print("A =\n", A)

print(type(A))              # numpy.ndarray (toujours)
print(type(A[0, 0]))        # numpy.int64
print(A.dtype)              # int64
print(A.shape)              # (3, 4) : 3 lignes, 4 colonnes
print(A.size)               # 12

print(A[:, 1])              # seconde colonne (indice 1)
print(A[2, :])              # troisieme ligne (indice 2)

# axis=0 parcourt les LIGNES -> resultat par COLONNE
# axis=1 parcourt les COLONNES -> resultat par LIGNE
print(np.mean(A))                    # moyenne totale ~ 1.583
print(np.mean(A, axis=0))            # [-0.333, 1.667, -0.333, 5.333]
print(np.mean(A, axis=1))            # [2.75, 0.5, 1.5]

moyenne_totale = np.mean(A)
print(f"La moyenne de tous les elements vaut {moyenne_totale:.2f}")
```

Question 6 : le piège de la division entière en place

Point de cours

```
# PIEGE : diviser une colonne d'entiers et reassigner DANS le meme
# tableau tronque silencieusement le resultat (sans erreur !).
A_piege = A.copy()
A_piege[0:2, :] *= 2
A_piege[:, -1] = A_piege[:, -1] / 3   # 2/3 -> 0 au lieu de 0.667 !
print(A_piege.dtype)    # int64 : toujours des entiers, silencieusement

# BONNE METHODE : convertir en float AVANT toute division.
A = A.astype(float)
A[0:2, :] = A[0:2, :] * 2
A[:, -1] = A[:, -1] / 3
print(A.dtype)    # float64
```

À partir d'ici, `A` désigne la matrice **modifiée** (float, 2 premières lignes doublées, dernière colonne divisée par 3) : les questions suivantes portent sur cette version, pas sur la matrice d'entiers d'origine.

Questions 7-9 : produit élément par élément vs matriciel, np.where

Point de cours

```
B = np.full((3, 4), 2)
B[:, 0] = 0

# '*' entre deux tableaux de meme forme : produit de HADAMARD
# (element par element), PAS un produit matriciel.
produit_elementwise = A * B

# Produit matriciel : nombre de colonnes du 1er = nombre de lignes
# du 2nd. A et B sont toutes deux 3x4 : A @ B est IMPOSSIBLE
# directement (ValueError). Deux solutions usuelles :
produit_3x3 = A @ B.T     # (3x4) @ (4x3) -> matrice 3x3
produit_4x4 = A.T @ B     # (4x3) @ (3x4) -> matrice 4x4

# np.where(condition, si_vrai, si_faux) : filtrage conditionnel
# element par element, plus lisible et plus rapide qu'une boucle.
CC = np.where(produit_elementwise > 5, produit_elementwise, 0)
```

1. Créer `A` avec `np.array`, afficher son type, le type d'un élément, et sa taille (`.shape`, `.size`).

   oup de pouc

   `type(A)` et `A.dtype` répondent à deux questions différentes.

   Correction

   type(A) -> numpy.ndarray ; A.dtype -> int64 ; A.shape -> (3, 4).

   Résultat Solution détaillée

   `type(A)` vaut `numpy.ndarray` (le type de l'objet), `A.dtype` vaut `int64` (le type des nombres stockés, car tous les éléments saisis sont des entiers), `A.shape` vaut $(3, 4)$ et `A.size` vaut 12.

2. Afficher la seconde colonne puis la troisième ligne de `A`. Puis calculer la moyenne de tous les éléments, par colonne, par ligne.

   oup de pouc

   Les indices commencent à 0 : « seconde colonne » = indice 1, « troisième ligne » = indice 2.

   Correction

   A[:, 1] et A[2, :] ; moyennes via np.mean(A, axis=0) et axis=1.

   Résultat Solution détaillée

   Seconde colonne (indice 1) : $[6, -1, 0]$; troisième ligne (indice 2) : $[-7, 0, 1, 12]$. Moyenne totale $\approx 1{,}583$; moyenne par colonne (`axis=0`) : $[-0{,}333,\ 1{,}667,\ -0{,}333,\ 5{,}333]$; moyenne par ligne (`axis=1`) : $[2{,}75,\ 0{,}5,\ 1{,}5]$.

3. Multiplier les deux premières lignes de `A` par 2, puis diviser sa dernière colonne par 3. Quel est le type des éléments de `A` après cette opération ? Que se passe-t-il si l'on oublie de convertir `A` en `float` au préalable ?

   Coup de pouce

   Testez la division par 3 sur la valeur 6 de la dernière colonne, puis sur une valeur qui ne tombe pas juste, comme 2.

   Correction

   Sans conversion, la division est tronquée silencieusement (int64) ; avec .astype(float) au préalable, on obtient bien float64.

Le piège et sa correction — **Solution détaillée**

Sans conversion préalable, `A[:, -1] = A[:, -1] / 3` sur un tableau `int64` tronque le résultat sans lever d'erreur : $12/3 = 4$ reste correct par coïncidence, mais $2/3$ devient 0 au lieu de $0{,}667$. Pour éviter ce piège, il faut convertir `A` en `float` *avant* toute division réassignée dans le tableau, avec `A = A.astype(float)`. Après conversion, `A.dtype` vaut bien `float64`.

4. Créer `B` (3×4, que des 2), remplacer sa première colonne par des 0. Que représente `A*B` ? Comment obtenir un véritable produit matriciel entre `A` et `B` ?

Coup de pouce

Vérifiez si les dimensions de `A` et `B` permettent un produit matriciel direct `A @ B`.

Correction

A*B est un produit élément par élément (Hadamard). A @ B est impossible (3x4 @ 3x4) ; utiliser A @ B.T ou A.T @ B.

Produit élément par élément vs matriciel — **Solution détaillée**

`A*B` effectue une multiplication terme à terme, valable car `A` et `B` ont la même forme $(3, 4)$: ce n'est pas un produit matriciel. Le produit matriciel classique nécessite que le nombre de colonnes du premier facteur soit égal au nombre de lignes du second : `A` et `B` étant toutes deux 3×4, `A @ B` est impossible directement. On peut néanmoins calculer `A @ B.T` (donne une matrice 3×3) ou `A.T @ B` (donne une matrice 4×4).

5. Créer `CC`, une matrice qui ne conserve que les éléments de `A*B` supérieurs à 5, et force les autres à 0.

Coup de pouce

`np.where(condition, valeur_si_vrai, valeur_si_faux)` est l'outil adapté à ce filtrage conditionnel élément par élément.

Correction

CC = np.where(A*B > 5, A*B, 0)

Utilisation de np.where — **Solution détaillée**

`CC = np.where(produit_elementwise > 5, produit_elementwise, 0)` calcule, pour chaque élément de `A*B`, la valeur elle-même si elle dépasse 5, et 0 sinon — un filtrage à la fois plus lisible et plus rapide qu'une double boucle `for` explicite. Ce calcul utilise `A` déjà modifiée par la question précédente (float, lignes doublées, dernière colonne divisée par 3).

Ne pas remarquer la troncature entière — **Remarques pédagogiques (écueils)**

Beaucoup d'étudiants ne remarqueront pas que leur division a été tronquée si le résultat « tombe juste » par coïncidence (par ex. $12/3 = 4$). Choisir un exemple en séance où la troncature change visiblement le résultat.

Confondre axis=0 et axis=1 — **Remarques pédagogiques (écueils)**

Erreur d'inversion très fréquente. Moyen mnémotechnique : « axis=0 fait disparaître la première dimension (les lignes), donc on obtient une valeur par colonne ».

Confondre * et @ (réflexe Matlab) — **Remarques pédagogiques (écueils)**

Confusion quasi systématique chez les étudiants venant d'autres langages (Matlab, où `*` est le produit matriciel par défaut) : insister particulièrement ici.

Utiliser A @ B directement — **Remarques pédagogiques (écueils)**

`A @ B` lève `ValueError` (dimensions incompatibles 3×4 @ 3×4). Faire chercher la condition de compatibilité (nombre de colonnes = nombre de lignes) et proposer `A @ B.T` ou `A.T @ B`.

Trois règles à retenir — **Bilan**

- Convertir en `float` avant toute division réassignée dans un tableau d'entiers.
- `axis=0` → résultat par colonne ; `axis=1` → résultat par ligne.
- `*` est toujours terme à terme ; le produit matriciel s'écrit `@`.

• 25 min

Exercice 7. Équation du second degré : réels puis complexes *(facultatif)*

★★☆☆☆

Exercice facultatif, fonctions, conditions et complexes

◁ Plan de la séance

On veut résoudre l'équation $ax^2 + bx + c = 0$.

1. Écrire une fonction `racines_reelles(a, b, c)` pour a, b, c réels, qui renvoie les racines réelles (aucune, une ou deux).
2. Écrire une fonction `racines(a, b, c)` qui renvoie toutes les racines dans $\mathbb{C}$ (module `cmath`), y compris lorsque $\Delta < 0$.
3. Faire fonctionner cette fonction avec a, b, c eux-mêmes complexes, et traiter le cas $a = 0$.
4. Vérifier les résultats en calculant $|ax^2 + bx + c|$ pour chaque racine trouvée.
5. Écrire une version vectorisée avec Numpy, résolvant plusieurs équations d'un coup.

Discriminant et racines — Prérequis

Pour $a \neq 0$, on pose $\Delta = b^2 - 4ac$. Les racines sont $x_\pm = \dfrac{-b \pm \sqrt{\Delta}}{2a}$, où $\sqrt{\Delta}$ désigne n'importe quelle racine carrée complexe de Δ. Si $\Delta = 0$, la racine est double. Cette formule reste valable pour a, b, c complexes.

Complexes en Python — Prérequis

Python note j l'unité imaginaire : `1j`, `3 + 2j`. `math.sqrt` refuse un nombre négatif, alors que `cmath.sqrt` accepte tout complexe.

Distinguer les cas ($\Delta > 0$, $= 0$, < 0, $a = 0$) avec `if/elif/else`.

Utiliser `cmath` et comprendre que la même formule marche pour des coefficients complexes.

Exercice facultatif

Réinvestit fonctions, conditions et vectorisation. La question 5 peut être laissée aux plus rapides.

Cas réel — Point de cours

```
def racines_reelles(a, b, c):
    """a, b, c reels avec a != 0 : renvoie les racines reelles (0, 1 ou 2)."""
    delta = b**2 - 4 * a * c
    if delta > 0:
        return ((-b - delta**0.5) / (2 * a), (-b + delta**0.5) / (2 * a))
    elif delta == 0:
        return (-b / (2 * a),)
    else:
        return ()      # pas de racine reelle
```

Cas complexe général et vérification — **Point de cours**

```
import cmath

def racines(a, b, c, tol=1e-12):
    """a, b, c reels OU complexes : renvoie toutes les racines complexes."""
    if abs(a) < tol:                     # equation du premier degre
        if abs(b) < tol:
            raise ValueError("equation degeneree : a = b = 0")
        return (-c / b,)
    delta = b**2 - 4 * a * c
    if abs(delta) < tol:                 # racine double
        return (-b / (2 * a),)
    s = cmath.sqrt(delta)                # racine carree complexe
    return ((-b - s) / (2 * a), (-b + s) / (2 * a))

def verifie(a, b, c, rac):
    """Residu |a x^2 + b x + c| pour chaque racine (doit etre ~ 0)."""
    return [abs(a * x**2 + b * x + c) for x in rac]
```

Le test `abs(delta) < tol` remplace `delta == 0` : avec des flottants, une égalité exacte est trop fragile.

Jeu de tests — **Point de cours**

```
tests = [(1, -3, 2),                   # deux racines reelles : 1 et 2
         (1, 2, 1),                    # racine double : -1
         (1, 0, 1),                    # racines imaginaires pures : +-i
         (1, -(3 + 1j), 2 + 2j),       # coefficient complexe : 2 et 1+i
         (1j, 1 - 3j, -2 + 2j),        # a complexe : memes racines
         (0, 2, -4)]                   # a = 0 : x = 2
for a, b, c in tests:
    r = racines(a, b, c)
    print((a, b, c), r, max(verifie(a, b, c, r)))
```

Version vectorisée avec Numpy — **Point de cours**

```
import numpy as np

def racines_np(a, b, c):
    """Version vectorisee : a, b, c tableaux Numpy de meme forme."""
    a, b, c = (np.asarray(t, dtype=complex) for t in (a, b, c))
    delta = b**2 - 4 * a * c
    s = np.sqrt(delta)                   # np.sqrt gere le complexe
    return (-b - s) / (2 * a), (-b + s) / (2 * a)

A = np.array([1, 1, 1]); B = np.array([-3, 2, 1]); C = np.array([2, 1, 1])
r1, r2 = racines_np(A, B, C)

# Comparaison avec la fonction toute faite de Numpy :
print(np.roots([1j, 1 - 3j, -2 + 2j]))
```

1. Écrire `racines_reelles(a, b, c)` et la tester sur $x^2 - 3x + 2$, $x^2 + 2x + 1$ et $x^2 + x + 1$.

oup de pou

Trois cas selon le signe de Δ ; la racine carrée d'un réel positif s'écrit `delta**0.5`.

Correction

Racines 1 et 2 ; racine double -1 ; aucune racine reelle.

Implémentation **ution détai**

Voir le code ci-dessus.

2. Écrire `racines(a, b, c)` avec `cmath`, et la tester sur x^2+1 et x^2+x+1.

oup de pouc

Remplacer `delta**0.5` par `cmath.sqrt(delta)` : plus besoin de distinguer le signe de Δ.

Correction

x**2 + 1 : racines +i et -i ; x**2 + x + 1 : (-1 +- i*sqrt(3))/2.

Résultat **Solution détaillée**

Pour x^2+1 : $\pm i$. Pour x^2+x+1 : $\dfrac{-1\pm i\sqrt{3}}{2}$, soit environ $-0{,}5 \pm 0{,}866\,i$.

3. Tester avec des coefficients complexes : $x^2-(3+i)x+(2+2i)=0$, puis la même équation multipliée par i. Que constate-t-on ? Traiter aussi $a=0$.

Coup de pouce

Multiplier une équation par un complexe non nul ne change pas ses racines. Pour $a=0$, l'équation est du premier degré.

Correction

Racines 2 et 1+i dans les deux cas ; pour a = 0 : x = -c/b.

Interprétation **Solution détaillée**

Les deux équations ont les mêmes racines, 2 et $1+i$. Pour $a=0$, on renvoie $-c/b$ (et on lève une erreur si aussi $b=0$).

4. Vérifier chaque solution en calculant $|ax^2+bx+c|$.

oup de pouc

Le résultat doit être nul à l'erreur d'arrondi près (de l'ordre de 10^{-16}).

Correction

Residus de l'ordre de 1e-16 ou nuls.

Vérification **ution détail**

Voir `verifie` ci-dessus : les résidus sont nuls ou de l'ordre de 10^{-16}, ce qui valide les racines.

5. Écrire une version vectorisée résolvant trois équations à la fois avec des tableaux Numpy.

Coup de pouce

Convertir les entrées en complexes avec `dtype=complex` pour que `np.sqrt` accepte les discriminants négatifs.

Correction

np.sqrt sur un tableau complexe ; comparer avec np.roots.

Implémentation **Solution détaillée**

Voir `racines_np` ci-dessus. Sans `dtype=complex`, `np.sqrt` renverrait `nan` pour un discriminant négatif.

`math.sqrt(-1)` lève une `ValueError` : il faut `cmath.sqrt`.

`-b + s / 2 * a` n'est pas `(-b + s) / (2 * a)` : la priorité des opérateurs donne un résultat faux sans erreur.

Un discriminant calculé peut valoir 10^{-16} au lieu de 0 : utiliser une tolérance.

Avec $a=0$, la division par $2a$ provoque une `ZeroDivisionError`.

Bilan

La formule du discriminant est la même pour des coefficients réels ou complexes ; seule la racine carrée doit être celle des complexes. Toujours vérifier une solution en la réinjectant dans l'équation.

Aller plus loin

Comparer votre fonction à `np.roots([a, b, c])` sur des milliers de coefficients aléatoires, et tracer l'écart maximal.

Bilan et prolongements du TD